NEXT ENGINEER · _SHEERAT_

Computer Graphics

BCS053

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 5 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards

UNIT 1 Display Devices and Output Primitives

UNIT 2 2D Transformations, Windowing and Clipping

UNIT 3 Three-Dimensional Graphics

UNIT 4 Curves and Surfaces

UNIT 5 Hidden Surfaces and Illumination

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables, matrices and algorithm steps.
3. **Solved examples**: the numericals that always appear (DDA, Bresenham, midpoint circle, transformations, Cohen-Sutherland, Liang-Barsky, window-to-viewport, Bezier, illumination).
4. **Formulas and pseudocode to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1. Raster vs random scan, frame buffer size, then work DDA, Bresenham and midpoint circle by hand once each.
- **Afternoon (3 hours):** Unit 2 (the biggest unit): all 2D matrices in homogeneous form, a composite transformation, window-to-viewport, Cohen-Sutherland and Liang-Barsky on the same line, Sutherland-Hodgman. Then Unit 3: 3D matrices and projections.
- **Evening (2 hours):** Units 4 and 5. Bezier formula with one point computed, B-spline properties, back-face test, z-buffer steps, illumination equation. Then the "Last-hour checklist".
- **In the exam:** write the algorithm steps first, then a table of iterations (k, pk, x, y). For matrices, always write the 3 × 3 (or 4 × 4) homogeneous matrix and multiply in the correct order.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Display Devices and Output Primitives

✍️ 2-mark definitions

- **Computer graphics:** creating, storing and manipulating pictures and models of objects using a computer.
- **Types of computer graphics:** interactive vs non-interactive (passive), and raster (pixel-based) vs vector (line-based).
- **Pixel:** the smallest addressable picture element on a raster screen.
- **Resolution:** the number of pixels that can be displayed, e.g. 1920 × 1080.
- **Aspect ratio:** ratio of the number of vertical points to horizontal points needed to draw equal-length lines (or width to height of the screen).
- **Frame buffer (refresh buffer):** memory that holds the intensity or colour value of every pixel on the screen.
- **Bit depth / bit planes:** number of bits per pixel; n bits give 2^n colours or grey levels.
- **Video controller:** hardware that reads the frame buffer and refreshes the screen, generating horizontal and vertical sweeps.
- **Refresh rate:** number of times per second the picture is redrawn (e.g. 60 Hz).
- **Interlacing:** drawing odd lines in one pass and even lines in the next, to reduce flicker at lower refresh rates.
- **Random-scan (vector) display:** the electron beam draws only the lines of the picture, one at a time, from a display list.
- **Raster-scan display:** the beam sweeps across every row from top to bottom, and intensities come from the frame buffer.
- **Scan conversion (rasterization):** converting a geometric primitive (line, circle) into a set of pixels.
- **Aliasing:** the jagged (staircase) appearance of lines due to discrete pixels; anti-aliasing smooths it.

💡 Core concepts

Raster scan vs random scan

Point	Raster scan	Random scan
Beam movement	whole screen, row by row	only along the lines of the picture
Picture stored as	intensity of every pixel in frame buffer	line-drawing commands in a display list
Resolution	lower; jagged lines	higher; smooth lines
Realistic images	yes, shading and colour fill	difficult
Cost	cheaper	expensive
Refresh	60 to 80 Hz regardless of picture	depends on number of lines

Point	Raster scan	Random scan
Example	TV, monitors	pen plotters, early CAD displays

Frame buffer size = horizontal resolution × vertical resolution × bits per pixel / 8 bytes.

Line drawing algorithms

Point	DDA	Bresenham
Arithmetic	floating point	integer only
Operations	multiplication/division and rounding	addition, subtraction, shift
Speed	slower	faster
Accuracy	rounding errors accumulate	more accurate

DDA steps: dx = x2 − x1, dy = y2 − y1, steps = max(|dx|, |dy|), x_inc = dx/steps, y_inc = dy/steps; plot round(x), round(y) and add the increments steps times.

Bresenham line (0 < m < 1): p0 = 2dy − dx. At each step x increases by 1. If pk < 0, plot (x + 1, y) and p(k+1) = pk + 2dy; otherwise plot (x + 1, y + 1) and p(k+1) = pk + 2dy − 2dx.

Midpoint circle: start at (0, r), p0 = 1 − r (or 5/4 − r). If pk < 0: next point (x + 1, y), p(k+1) = pk + 2x(k+1) + 1. Otherwise: next point (x + 1, y − 1), p(k+1) = pk + 2x(k+1) + 1 − 2y(k+1). Stop when x ≥ y. Use 8-way symmetry: (±x, ±y), (±y, ±x) plus centre offset.

Parallel line and circle algorithms: with several processors, a line can be split into segments (each processor gets a start point and runs Bresenham on its part), or each processor handles one row or column of the bounding box and tests the distance from the line. Circles can be split by octant, one processor per octant.

🧮 Solved examples

Example 1 (frame buffer). 1024 × 768 at 24 bits per pixel: 1024 × 768 × 3 = 2,359,296 bytes ≈ **2.25 MB**. At 8 bits per pixel: 786,432 bytes = 768 KB.

Example 2 (DDA). Line from (2, 3) to (8, 6): dx = 6, dy = 3, steps = 6, x_inc = 1, y_inc = 0.5.

k	x	y	Plotted
0	2	3	(2, 3)
1	3	3.5	(3, 4)
2	4	4	(4, 4)
3	5	4.5	(5, 5)
4	6	5	(6, 5)
5	7	5.5	(7, 6)
6	8	6	(8, 6)

(Rounding 0.5 upward.)

Example 3 (Bresenham). Line from (20, 10) to (30, 18): dx = 10, dy = 8, 2dy = 16, 2dy − 2dx = −4, p0 = 2dy − dx = 6.

k	pk	Next point
0	6	(21, 11)
1	2	(22, 12)
2	−2	(23, 12)
3	14	(24, 13)
4	10	(25, 14)
5	6	(26, 15)
6	2	(27, 16)
7	−2	(28, 16)
8	14	(29, 17)
9	10	(30, 18)

Example 4 (midpoint circle, r = 10, centre origin). p0 = 1 − 10 = −9.

k	pk	(x(k+1), y(k+1))	2x(k+1)	2y(k+1)
0	−9	(1, 10)	2	20
1	−6	(2, 10)	4	20
2	−1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	−3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14

Stop at (7, 7) since x = y. Reflect into the other 7 octants.

Pseudocode to remember

```
BRESENHAM(x1, y1, x2, y2)      // 0 < slope < 1, x1 < x2
  dx = x2 - x1; dy = y2 - y1
  p = 2*dy - dx; x = x1; y = y1
  plot(x, y)
  while x < x2
     x = x + 1
     if p < 0: p = p + 2*dy
     else:     y = y + 1; p = p + 2*dy - 2*dx
     plot(x, y)

MIDPOINT-CIRCLE(xc, yc, r)
  x = 0; y = r; p = 1 - r
  while x <= y
     plot the 8 symmetric points (xc ± x, yc ± y), (xc ± y, yc ± x)
     x = x + 1
     if p < 0: p = p + 2*x + 1
     else:     y = y - 1; p = p + 2*x + 1 - 2*y
```

⭐ Important questions

1. Compare raster-scan and random-scan displays; explain the frame buffer and video controller.
2. Explain the DDA algorithm and draw a line between given points.
3. Derive Bresenham's line algorithm and apply it to given end points.
4. Explain the midpoint circle algorithm and compute points for a given radius.
5. Explain types of computer graphics, applications and parallel line algorithms.

UNIT 2 2D Transformations, Windowing and Clipping

✍️ 2-mark definitions

- **Transformation:** a change in position, size, orientation or shape of an object.
- **Homogeneous coordinates:** representing (x, y) as (x, y, 1) so that translation, like rotation and scaling, can be written as matrix multiplication.
- **Composite transformation:** a sequence of transformations combined into one matrix by multiplying matrices (applied right to left).
- **Reflection:** a mirror image of an object about an axis or line.
- **Shearing:** a transformation that slants the object, e.g. $x' = x + sh_x \cdot y$.
- **Window:** the region of the world coordinate scene selected for display.
- **Viewport:** the area on the display device where the window is mapped.
- **Viewing transformation (window-to-viewport mapping):** mapping world coordinates in the window to device coordinates in the viewport.
- **Clipping:** removing the parts of a picture outside the clip window.
- **Region code (outcode):** a 4-bit code (Top, Bottom, Right, Left) telling which side of the window a point is on, used in Cohen-Sutherland.
- **Polygon clipping:** clipping a closed polygon so the result is again a closed polygon (Sutherland-Hodgman, Weiler-Atherton).
- **Text clipping:** all-or-none string, all-or-none character, or individual character-part clipping.

💡 Core concepts

2D homogeneous matrices (column vector $P' = M \cdot P$)

Transformation	Matrix (rows)
Translation (tx, ty)	[1 0 tx; 0 1 ty; 0 0 1]
Scaling (sx, sy)	[sx 0 0; 0 sy 0; 0 0 1]
Rotation θ (counter-clockwise)	[cos θ −sin θ 0; sin θ cos θ 0; 0 0 1]
Reflection about x-axis	[1 0 0; 0 −1 0; 0 0 1]
Reflection about y-axis	[−1 0 0; 0 1 0; 0 0 1]
Reflection about origin	[−1 0 0; 0 −1 0; 0 0 1]
Reflection about y = x	[0 1 0; 1 0 0; 0 0 1]
Reflection about y = −x	[0 −1 0; −1 0 0; 0 0 1]
x-shear	[1 shx 0; 0 1 0; 0 0 1]
y-shear	[1 0 0; shy 1 0; 0 0 1]

Rotation or scaling about a fixed point (xf, yf): translate (xf, yf) to the origin, rotate or scale, translate back: $M = T(xf, yf) \cdot R(\theta) \cdot T(-xf, -yf)$.

Reflection about an arbitrary line y = mx + c: translate by (0, −c), rotate by $-\theta$ ($\theta = \tan^{-1} m$) to align with the x-axis, reflect about the x-axis, rotate back by θ, translate back by (0, c).

Viewing pipeline: modelling coordinates → world coordinates → viewing coordinates → normalized coordinates (window to viewport) → device coordinates.

Window-to-viewport mapping: $xv = xvmin + (xw - xwmin) \cdot sx$, $yv = yvmin + (yw - ywmin) \cdot sy$, where $sx = (xvmax - xvmin)/(xwmax - xwmin)$ and $sy = (yvmax - yvmin)/(ywmax - ywmin)$.

Cohen-Sutherland line clipping

1. Give each end point a 4-bit code TBRL (bit set if the point is above, below, right of, left of the window).
2. Both codes 0000: trivially accept.
3. Logical AND of the codes non-zero: trivially reject (both on the same outside side).
4. Otherwise, pick an outside end point, find the intersection with the boundary indicated by its code, replace the point, recompute the code and repeat. - Intersections: with x = xmin or xmax, $y = y1 + m(x - x1)$; with y = ymin or ymax, $x = x1 + (y - y1)/m$.

Liang-Barsky line clipping. Parametric line $x = x1 + u \cdot dx$, $y = y1 + u \cdot dy$, $0 \le u \le 1$. Define $p1 = -dx$, $q1 = x1 - xmin$; $p2 = dx$, $q2 = xmax - x1$; $p3 = -dy$, $q3 = y1 - ymin$; $p4 = dy$, $q4 = ymax - y1$.

- If pk = 0 and qk < 0, the line is parallel and outside: reject.
- For pk < 0 (entering), u1 = max(0, qk/pk). For pk > 0 (leaving), u2 = min(1, qk/pk).
- If u1 > u2, reject; else the clipped end points are at u1 and u2. It is faster than Cohen-Sutherland (fewer intersection calculations).

Clipping against non-rectangular (convex) windows: Cyrus-Beck uses the parametric line and the inward normals of each edge of a convex polygon window, keeping the largest entering and smallest leaving u.

Sutherland-Hodgman polygon clipping: clip the polygon against each window edge in turn. For each edge from vertex S to vertex P:

1. S inside, P inside: output P.
2. S inside, P outside: output the intersection I.
3. S outside, P outside: output nothing.
4. S outside, P inside: output I and P.

It works for convex clip windows; with concave polygons it can leave extra edges.

Weiler-Atherton polygon clipping: walks along the subject polygon and the clip window boundaries, switching at intersection points (entering and leaving); it handles concave polygons correctly and can produce several separate polygons.

Curve clipping: use bounding rectangles first (accept or reject the whole curve), then compute intersections of the curve with the window edges.

🧮 Solved examples

Example 1 (rotation). Rotate P(4, 3) by 90° counter-clockwise about the origin: x′ = 4 cos 90° − 3 sin 90° = −3, y′ = 4 sin 90° + 3 cos 90° = 4. **P′ = (−3, 4)**.

Example 2 (scaling about a fixed point). Scale the square A(2, 2), B(4, 2), C(4, 4), D(2, 4) by 2 about A(2, 2): x′ = 2 + 2(x − 2), y′ = 2 + 2(y − 2). Result A(2, 2), B(6, 2), C(6, 6), D(2, 6).

Example 3 (composite: rotate 45° about (1, 1) the point (3, 1)). Translate to origin: (2, 0). Rotate 45°: (2 cos 45°, 2 sin 45°) = (1.414, 1.414). Translate back: **(2.414, 2.414)**.

Example 4 (window to viewport). Window (10, 10) to (50, 40); viewport (0, 0) to (200, 150). sx = 200/40 = 5, sy = 150/30 = 5. Point (30, 25) maps to (0 + 20 × 5, 0 + 15 × 5) = **(100, 75)**.

Example 5 (Cohen-Sutherland). Window xmin = 10, xmax = 50, ymin = 10, ymax = 40. Line P1(5, 20) to P2(60, 30).

- Codes: P1 = 0001 (left), P2 = 0010 (right). AND = 0000, so neither accept nor reject.
- m = 10/55 ≈ 0.1818.
- Clip P1 at x = 10: y = 20 + 0.1818 × 5 = 20.91.
- Clip P2 at x = 50: y = 20 + 0.1818 × 45 = 28.18.
- Visible segment: **(10, 20.91) to (50, 28.18)**.

Example 6 (Liang-Barsky on the same line). dx = 55, dy = 10.

- p1 = −55, q1 = −5 → u = 0.0909 (entering).
- p2 = 55, q2 = 45 → u = 0.818 (leaving).
- p3 = −10, q3 = 10 → u = −1 (entering).
- p4 = 10, q4 = 20 → u = 2 (leaving).
- u1 = max(0, 0.0909, −1) = 0.0909; u2 = min(1, 0.818, 2) = 0.818.
- Points: (5 + 55 × 0.0909, 20 + 10 × 0.0909) = (10, 20.91) and (5 + 55 × 0.818, 20 + 8.18) = (50, 28.18). Same answer as Cohen-Sutherland.

Formulas to remember

- Rotation: x′ = x cos θ − y sin θ, y′ = x sin θ + y cos θ.
- Fixed-point scaling: x′ = xf + sx(x − xf).
- Window to viewport: xv = xvmin + (xw − xwmin)(xvmax − xvmin)/(xwmax − xwmin).
- Region code bits: TBRL (1000 top, 0100 bottom, 0010 right, 0001 left).

⭐ Important questions

1. Explain basic 2D transformations with homogeneous matrices; why are homogeneous coordinates used?
2. Derive the composite matrix for rotation about an arbitrary point or reflection about an arbitrary line; apply it to a triangle.
3. Explain the viewing pipeline and derive the window-to-viewport transformation.
4. Explain Cohen-Sutherland and Liang-Barsky line clipping and apply them to a given line.

NEXT ENGINEER _sheerat_

5. Explain Sutherland-Hodgman and Weiler-Atherton polygon clipping; explain curve and text clipping.

NEXT ENGINEER
sheerat

UNIT 3 Three-Dimensional Graphics

✍️ 2-mark definitions

- **3D geometric primitives:** basic 3D shapes such as points, lines, polygons, polyhedra, spheres, cylinders and cones.
- **Polygon surface (boundary) representation:** an object is described by a set of polygon faces, using vertex, edge and polygon tables.
- **Sweep representation:** a 3D object made by moving a 2D shape along a path (translational or rotational sweep).
- **Constructive solid geometry (CSG):** building solids by combining primitives with union, intersection and difference.
- **Octree:** a hierarchical tree that divides 3D space into eight octants recursively, used for solid modelling.
- **Projection:** mapping 3D points onto a 2D view plane.
- **Parallel projection:** projectors are parallel; it keeps relative sizes (orthographic and oblique).
- **Perspective projection:** projectors meet at a centre of projection; distant objects look smaller (realistic).
- **Vanishing point:** the point where parallel lines not parallel to the view plane appear to meet in perspective.
- **3D clipping:** removing parts outside a view volume (a rectangular box for parallel, a frustum for perspective).

💡 Core concepts

3D homogeneous matrices (4 × 4)

Transformation	Matrix (rows)
Translation	[1 0 0 tx; 0 1 0 ty; 0 0 1 tz; 0 0 0 1]
Scaling	[sx 0 0 0; 0 sy 0 0; 0 0 sz 0; 0 0 0 1]
Rotation about z	[cos θ −sin θ 0 0; sin θ cos θ 0 0; 0 0 1 0; 0 0 0 1]
Rotation about x	[1 0 0 0; 0 cos θ −sin θ 0; 0 sin θ cos θ 0; 0 0 0 1]
Rotation about y	[cos θ 0 sin θ 0; 0 1 0 0; −sin θ 0 cos θ 0; 0 0 0 1]
Reflection about xy-plane	[1 0 0 0; 0 1 0 0; 0 0 −1 0; 0 0 0 1]

Rotation about an arbitrary axis: translate so the axis passes through the origin, rotate it onto a coordinate axis (two rotations), rotate by θ, then apply the inverse rotations and translation.

Parallel vs perspective projection

Point	Parallel	Perspective
Projectors	parallel	converge at the centre of projection
Size	preserved	farther objects smaller
Realism	less	more
Use	engineering drawings	realistic scenes, games
Types	orthographic (top, front, side, isometric), oblique (cavalier, cabinet)	one-, two-, three-point

Oblique projections: $x_p = x + L\cos\varphi$, $y_p = y + L\sin\varphi$ with $L = z/\tan\alpha$. Cavalier ($\alpha = 45°$): depth lines keep full length. Cabinet ($\alpha \approx 63.4°$): depth lines are half length, looking more realistic.

Perspective projection (centre of projection at origin, view plane z = d): $x_p = x \cdot d/z$, $y_p = y \cdot d/z$.

3D viewing pipeline: modelling → world → viewing coordinates (set by view reference point, view-plane normal and view-up vector) → projection → normalized view volume → clipping → workstation transformation.

3D clipping: extend Cohen-Sutherland to 6-bit codes (left, right, bottom, top, near, far) against the view volume; normalize the perspective frustum to a canonical volume first.

🧮 Solved examples

Example 1 (3D translation then scaling). Point (2, 3, 4) translated by (1, −1, 2) gives (3, 2, 6); scaling by (2, 2, 0.5) then gives **(6, 4, 3)**.

Example 2 (rotation about z by 90°). (1, 0, 5) → (0, 1, 5).

Example 3 (perspective). View plane at d = 2, point (4, 6, 8): $x_p = 4 \times 2/8 = 1$, $y_p = 6 \times 2/8 = 1.5$. Projected point **(1, 1.5)**. A point twice as far, (4, 6, 16), projects to (0.5, 0.75), half the size.

Example 4 (cabinet projection). Unit cube corner (1, 1, 1), $\varphi = 45°$, $L = 0.5$: $x_p = 1 + 0.5 \times 0.707 = 1.354$, $y_p = 1.354$.

Formulas to remember

- Perspective: $x_p = xd/z$, $y_p = yd/z$.
- Oblique: $x_p = x + L\cos\varphi$, $y_p = y + L\sin\varphi$; cavalier $L = z$, cabinet $L = z/2$.
- Composite 3D rotation about an arbitrary axis: $T^{-1}\,Rx^{-1}\,Ry^{-1}\,Rz(\theta)\,Ry\,Rx\,T$.

⭐ Important questions

1. Explain 3D object representation methods (polygon tables, sweep, CSG, octrees).
2. Give the 3D transformation matrices; derive rotation about an arbitrary axis.
3. Explain the 3D viewing pipeline.
4. Explain parallel and perspective projections with types and matrices; compare them.

NEXT ENGINEER _sheerat_

5. Explain 3D clipping.

NEXT ENGINEER
sheerat

UNIT 4 Curves and Surfaces

✍️ 2-mark definitions

- **Quadric surface:** a surface described by a second-degree equation, e.g. sphere, ellipsoid, torus, paraboloid.
- **Sphere:** $x^2 + y^2 + z^2 = r^2$; parametric form x = r cos φ cos θ, y = r cos φ sin θ, z = r sin φ.
- **Ellipsoid:** $(x/rx)^2 + (y/ry)^2 + (z/rz)^2 = 1$.
- **Blobby object:** an object without a fixed shape (e.g. water drops, molecules), modelled as a sum of density functions f(x, y, z) = Σ bk e^(−ak rk²) − T = 0.
- **Spline:** a smooth curve made of polynomial pieces that pass through or near control points.
- **Control points:** points that determine the shape of a spline curve. The convex hull of the control points bounds the curve.
- **Interpolating vs approximating spline:** an interpolating spline passes through every control point; an approximating spline only follows them (Bezier, B-spline).
- **Parametric continuity:** C0 (curves meet), C1 (equal first derivatives), C2 (equal second derivatives) at joints.
- **Geometric continuity:** G0, G1, G2: directions are proportional, not necessarily equal.
- **Bezier curve:** an approximating curve defined by n + 1 control points using Bernstein polynomials.
- **B-spline curve:** an approximating curve built from basis functions over a knot vector; it gives local control and a degree independent of the number of control points.

💡 Core concepts

Bezier curve: $P(u) = \Sigma\ (i = 0 \text{ to } n)\ P_i \cdot C(n, i)\ u^i\ (1 - u)^{(n - i)},\ 0 \le u \le 1$.

- Cubic: $P(u) = (1 - u)^3 P0 + 3u(1 - u)^2 P1 + 3u^2(1 - u)P2 + u^3 P3$.

Properties of Bezier curves

- Passes through the first and last control points.
- Degree is one less than the number of control points.
- Lies within the convex hull of its control points.
- Tangent at the start is along P0P1 and at the end along P(n−1)Pn.
- No local control: moving one point changes the whole curve.
- Invariant under affine transformations.

Bezier vs B-spline

Point	Bezier	B-spline
Degree	fixed by number of points (n)	chosen independently (k − 1)
Local control	no	yes

Point	Bezier	B-spline
Passes through end points	yes	not in general (unless clamped knots)
Basis	Bernstein polynomials	B-spline basis over knot vector

B-spline types: uniform (equally spaced knots), open uniform (repeated end knots, so it passes through end points), non-uniform; NURBS add weights (rational) and can represent conics exactly.

Bezier surface: $P(u, v) = \Sigma_i \Sigma_j P_{ij} B(i, m)(u) B(j, n)(v)$, a tensor product of two Bezier curves over a grid of control points.

🧮 Solved examples

Example 1 (cubic Bezier point). P0(0, 0), P1(1, 2), P2(3, 2), P3(4, 0) at u = 0.5. Weights: $(1 - u)^3 = 1/8$, $3u(1 - u)^2 = 3/8$, $3u^2(1 - u) = 3/8$, $u^3 = 1/8$.

- x = (0 + 3 × 1 + 3 × 3 + 4)/8 = 16/8 = 2.
- y = (0 + 3 × 2 + 3 × 2 + 0)/8 = 12/8 = 1.5.
- **P(0.5) = (2, 1.5)**.

Example 2 (quadratic Bezier). P0(0, 0), P1(2, 4), P2(4, 0) at u = 0.5: weights 1/4, 1/2, 1/4. Point = (0 + 1 + 1, 0 + 2 + 0) = **(2, 2)**.

Example 3 (blending functions of a cubic Bezier). BEZ(0,3) = $(1 - u)^3$, BEZ(1,3) = $3u(1 - u)^2$, BEZ(2,3) = $3u^2(1 - u)$, BEZ(3,3) = u^3. They always add up to 1.

Formulas to remember

- Bernstein: B(i, n)(u) = C(n, i) u^i (1 − u)^(n − i).
- Sphere and ellipsoid equations above.
- B-spline: $P(u) = \Sigma P_i N(i, k)(u)$, with N(i, 1) = 1 if $u_i \le u < u(i+1)$ else 0, and the Cox-de Boor recursion for higher k.

⭐ Important questions

1. Explain quadric surfaces (sphere, ellipsoid) and blobby objects.
2. What are splines? Explain interpolation vs approximation and parametric continuity.
3. Derive the Bezier curve equation; state its properties; compute points for given control points.
4. Explain B-spline curves and compare them with Bezier curves.
5. Explain Bezier surfaces.

NEXT ENGINEER _sheerat_

UNIT 5 Hidden Surfaces and Illumination

✍️ 2-mark definitions

- **Hidden surface removal (visible surface detection):** finding which surfaces are visible from the viewpoint and removing the rest.
- **Object-space vs image-space method:** object-space methods compare objects and parts of objects (e.g. back-face); image-space methods decide visibility pixel by pixel (e.g. depth buffer).
- **Back-face detection:** a polygon faces away from the viewer if its normal N and view direction V satisfy $V \cdot N > 0$ (for viewing along $-z$, if $C \leq 0$ in $Ax + By + Cz + D = 0$ the face is a back face in a right-handed system).
- **Depth buffer (z-buffer):** an image-space method storing the depth of the nearest surface for every pixel.
- **A-buffer:** an extension of the z-buffer that keeps a list of surface fragments per pixel, supporting transparency and anti-aliasing.
- **Scan-line method:** processes one scan line at a time, using an edge table and active edge list to find the nearest surface along the line.
- **Illumination model (lighting model):** a formula for the light intensity reflected from a surface point.
- **Ambient light:** uniform background light from reflections in the scene.
- **Diffuse reflection:** light scattered equally in all directions from a dull surface (Lambert's cosine law).
- **Specular reflection:** shiny highlight seen near the mirror direction.
- **Phong model:** an empirical model for specular highlights using $(\cos \alpha)^n$, where n is the shininess exponent.
- **Warn model:** a lighting model that controls light direction and spread using a directed light source with flaps and cones, like studio spotlights.
- **Intensity attenuation:** decrease of light intensity with distance, $f(d) = 1/(a0 + a1\, d + a2\, d^2)$.

💡 Core concepts

Depth buffer algorithm

1. Set depth[x][y] = maximum depth (far) and frame[x][y] = background for every pixel.
2. For each polygon, for each pixel it covers, compute its depth z.
3. If z is nearer than depth[x][y], set depth[x][y] = z and frame[x][y] = surface colour.
4. Display the frame buffer.

Depth along a scan line can be found incrementally: $z' = z - A/C$ for the next pixel.

Point	Depth buffer	A-buffer	Scan-line
Space	image	image	image

Point	Depth buffer	A-buffer	Scan-line
Memory	frame + depth buffer	more (fragment lists)	only one line at a time
Transparency	no	yes	possible
Sorting	none	none	edges sorted by y

Basic illumination model

- Ambient: $I = k_a I_a$.
- Diffuse: $I = k_d I_l (N \cdot L)$.
- Specular (Phong): $I = k_s I_l (R \cdot V)^n$.
- Combined: $I = k_a I_a + \Sigma f(d) I_l [k_d (N \cdot L) + k_s (R \cdot V)^n]$.
- Using the halfway vector $H = (L + V)/|L + V|$: specular term $k_s I_l (N \cdot H)^n$.

Shading methods: constant (flat) shading uses one intensity per polygon; Gouraud shading interpolates intensities across the polygon (smooth but may miss highlights); Phong shading interpolates normals and computes intensity per pixel (best highlights, slower).

Colour considerations: the model is applied separately to R, G and B components with colour-specific coefficients (kdR, kdG, kdB).

Transparency: $I = (1 - k_t) I_{reflected} + k_t I_{transmitted}$, where k_t is the transparency coefficient. Refraction follows Snell's law.

Shadows: apply hidden-surface methods from the light source's position; surfaces not visible from the light are in shadow and receive only ambient light.

🧮 Solved examples

Example 1 (back-face test). Viewer looks along $V = (0, 0, -1)$. Face normal $N1 = (0, 0, 1)$: $V \cdot N1 = -1 < 0$, so it faces the viewer (visible). $N2 = (0, 0, -1)$: $V \cdot N2 = 1 > 0$, so it is a **back face** and is removed.

Example 2 (diffuse intensity). $I_l = 1$, $k_d = 0.8$, angle between N and L is 60°: $I = 0.8 \times 1 \times \cos 60° =$ **0.4**. With ambient $k_a = 0.2$, $I_a = 0.5$: total $= 0.1 + 0.4 =$ **0.5**.

Example 3 (specular). $k_s = 0.5$, $I_l = 1$, angle between R and V is 30°, $n = 10$: $\cos 30° = 0.866$; $0.866^{10} \approx 0.237$; $I = 0.5 \times 0.237 \approx$ **0.118**.

Example 4 (z-buffer). At pixel (5, 5) the buffer holds $z = 0.7$ (smaller z is nearer). A new surface gives $z = 0.4$ at that pixel, so it is nearer: update depth to 0.4 and colour to the new surface. A later surface with $z = 0.6$ is behind, so no change.

Formulas to remember

- $I = k_a I_a + f(d) I_l [k_d (N \cdot L) + k_s (R \cdot V)^n]$.
- $f(d) = 1/(a_0 + a_1 d + a_2 d^2)$.
- $R = 2(N \cdot L)N - L$.
- Transparency: $I = (1 - k_t) I_{refl} + k_t I_{trans}$.

NEXT ENGINEER _sheerat_

⭐ Important questions

1. Classify hidden-surface methods; explain back-face detection.
2. Explain the depth-buffer (z-buffer) algorithm with its advantages and disadvantages; compare with the A-buffer.
3. Explain the scan-line method of visible surface detection.
4. Explain the basic illumination model: ambient, diffuse, specular and the Phong model; compute intensities.
5. Explain the Warn model, intensity attenuation, colour, transparency and shadows; compare Gouraud and Phong shading.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Raster vs random scan table; frame buffer size formula (2.25 MB example).
- ☐ DDA steps; Bresenham p0 = 2dy − dx and updates; (20, 10) to (30, 18) table.
- ☐ Midpoint circle p0 = 1 − r; r = 10 table; 8-way symmetry.
- ☐ 2D matrices in homogeneous form; fixed-point rotation $T\,R\,T^{-1}$; reflection about y = x.
- ☐ Window-to-viewport formula; viewing pipeline.
- ☐ Cohen-Sutherland TBRL codes; trivial accept/reject; intersection formulas.
- ☐ Liang-Barsky p and q values; u1 = max entering, u2 = min leaving.
- ☐ Sutherland-Hodgman 4 cases; Weiler-Atherton for concave.
- ☐ 3D rotation matrices; perspective x d/z; cavalier vs cabinet.
- ☐ Bezier equation and properties; (2, 1.5) example; B-spline local control.
- ☐ Back-face V · N > 0; z-buffer steps; A-buffer; scan-line.
- ☐ Illumination equation; Gouraud vs Phong shading; attenuation; transparency.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Computer Graphics: C Version	Donald Hearn, M. Pauline Baker	Pearson Education	WorldCat
2	Computer Graphics: Principles and Practice	Foley, van Dam, Feiner, Hughes	Pearson Education	WorldCat
3	Procedural Elements for Computer Graphics	David F. Rogers	McGraw Hill	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
4	Principles of Interactive Computer Graphics	W. M. Newman, R. F. Sproull	Tata McGraw-Hill	WorldCat
5	Computer Graphics	Amarendra N. Sinha, Arun D. Udai	Tata McGraw-Hill	WorldCat
6	Computer Graphics	R. K. Maurya	Wiley Dreamtech	WorldCat
7	Fundamentals of Computer Graphics and Multimedia	D. P. Mukherjee	PHI Learning	WorldCat
8	Computer Graphics with OpenGL	Donald Hearn, M. Pauline Baker	Pearson Education	WorldCat

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER _sheerat_